

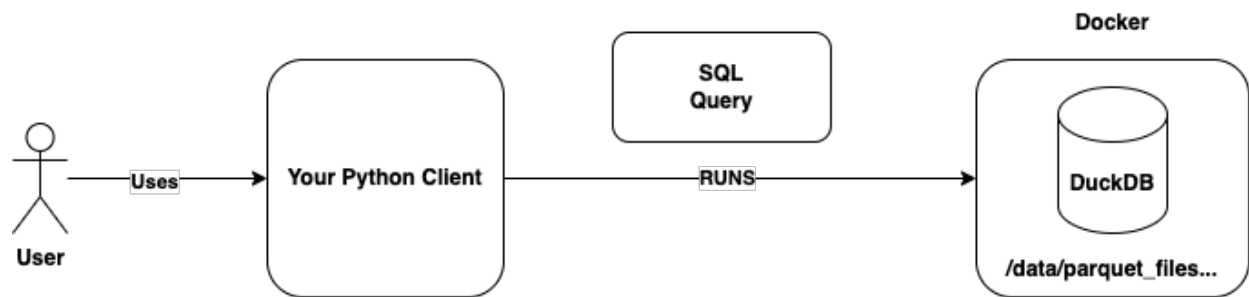
Analytics Team – Technical Interview

Background

You are provided with a dataset in parquet format, containing records from drives with manual detection labels. The data is split across multiple parquet files and has the following schema:

- **clip_name** (str): Name of the recorded clip.
- **frame_id** (int): Frame index.
- **vehicle_type** (str): Type of vehicle (e.g., car, bike, truck, etc.).
- **detection** (bool): Whether the vehicle was detected by the algorithm.
- **distance** (int): Distance to the vehicle (1–100).

Your goal is to develop a Python client that calculates and exposes the success rate of detections for different vehicle types and distance bins, using DuckDB for SQL-based analysis.



1. Step 1 – Initial setup

- Create a basic setup for running queries on DuckDB with Python code.
- Connect/import the data (parquets) and create a table `interview_table` for further querying.
- DuckDB must run as a Docker container. You may use any Docker image for DuckDB. You can also start with DuckDB in in-memory mode and move to Docker once you finish the whole task.
- At the end of this step, you should be able to run a basic query like `SELECT * FROM interview_table LIMIT 10` and see the first ten rows of the data.

2. Step 2 - Create a query over the data

- Create an SQL query that will produce the following result:

Distance	1-10	11-20		91-100
----------	------	-------	------	--------

Car	%True	%True		%True
Truck	%True	%True		%True
Bicycle	%True	%True		%True
Other vehicle type				
...				
...				

3. Step 3 - Create Python wrapper for the query

- Create a Python client that will provide this query as an Python API. Assume that query should be triggered by users who wish to use the results but have no knowledge in SQL and require some Python wrapper to use it.
- Expose the relevant parameters of the query that the user can change (e.g., the bin's start and end values).
- Make sure your implementation has the following aspects:
 - a. The project is well structured.
 - b. The code is well tested.
 - c. Includes readme.
 - d. Includes setup.
 - e. Contains docstrings.
 - f. Contains type hints.
 - g. Has a clear entry point for the users.
 - h. Allows easy development of additional queries and modularity is required.
 - i. Handles exceptions and edge cases which are clear to the user.
 - j. Uses third-party libraries.

Good Luck!